

Enterprise Database Infrastructure Administration Lab

Design, Deployment, Security Hardening, Backup, Replication and
Monitoring of PostgreSQL Infrastructure on Linux

David Gottlieb SITTI
<https://www.linkedin.com/in/davidgottliebsitti/>

SUMMARY

BACKUP & RECOVERY STRATEGY.....	2
---------------------------------	---

BACKUP & RECOVERY STRATEGY

I. Objective

The purpose of this deliverable is to implement a professional backup and recovery strategy for the PostgreSQL environment.

A database administrator is not judged solely on their ability to deploy databases, but also on their ability to recover data when something goes wrong.

At the end of this phase, you will have :

- Logical backups
- Physical backups
- Automated backup jobs
- Backup retention policy
- Recovery procedures
- Disaster recovery documentation
- Backup validation process
- Recovery testing

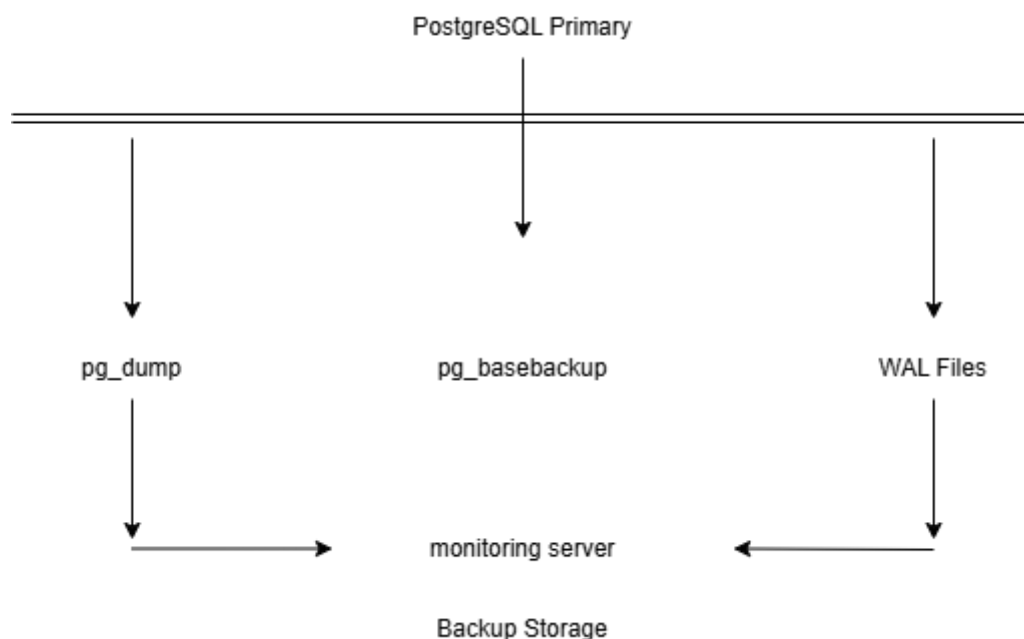
II. Backup Strategy Design

A professional environment never relies on a single backup method.

We will implement three layers.

Layer	Method	Purpose
Level 1	pg_dump	Database-level backup
Level 2	pg_basebackup	Cluster-level backup
Level 3	WAL Archiving	Point-in-Time Recovery

III. Backup Architecture



IV. Create Backup Storage

Perform on monitoring.

Create directory structure :

```
sudo mkdir -p /backups/postgresql
sudo mkdir -p /backups/postgresql/logical
sudo mkdir -p /backups/postgresql/physical
sudo mkdir -p /backups/postgresql/wal
```

V. Configure Backup Operator

On pg-primary.

Connect :

```
sudo -u postgres psql
```

Grant backup permissions :

```
ALTER ROLE backup_operator
WITH REPLICATION;
```

Verify :

```
\du
postgres=# ALTER ROLE backup_operator
WITH REPLICATION;
postgres=#
postgres=#
postgres=# \du
List of roles
-----
Role name | Attributes
-----
app_user  |
backup_operator | Replication
dba_admin | Superuser, Create role, Create DB
postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS
postgres=#
```

VI. Logical Backup Implementation (pg_dump)

Logical backups are used for :

- Accidental deletion
- Data migration
- Development environments
- Individual database recovery

Manual Backup Test

On pg-primary :

```
mkdir -p ~/db_backups
```

Create backup :

```
pg_dump \  
-h localhost \  
-F c \  
company_prod \  
> ~/db_backups/company_prod.backup
```

Verify :

```
ls -lh ~/db_backups  
  
root@pg-primary:~# pg_dump -U dba_admin -h localhost -F c company_prod > /db_backups/company_prod.backup  
Password:  
root@pg-primary:~# ls -lh /db_backups/  
total 20K  
-rw-r--r-- 1 root root 19K juin 14 01:34 company_prod.backup  
root@pg-primary:~# cd /db_backups/  
root@pg-primary:/db_backups# ll  
total 28  
drwxr-xr-x 2 root root 4096 juin 14 01:32 ./  
drwxr-xr-x 24 root root 4096 juin 14 01:29 ../  
-rw-r--r-- 1 root root 19123 juin 14 01:34 company_prod.backup  
root@pg-primary:/db_backups# nano company_prod.backup  
root@pg-primary:/db_backups#  
root@pg-primary:/db_backups#
```

VII. Transfer Backup to Monitoring Server

Copy backup :

```
scp \  
~/db_backups/company_prod.backup \  
david@monitoring:/backups/postgresql/logical/
```

Verify :

```
ssh david@monitoring  
ls -lh /backups/postgresql/logical/  
  
root@monitoring:~# ls -lh /backups/postgresql/logical/  
total 20K  
-rw-r--r-- 1 root root 19K juin 14 12:39 company_prod.backup  
root@monitoring:~#
```

VIII. Backup Automation Script

Create a script on pg-primary.

```
mkdir -p ~/scripts  
nano ~/scripts/postgres_logical_backup.sh
```

Content :

```
#!/bin/bash  
  
DATE=$(date +%F)  
  
pg_dump -U dba_admin -h localhost -F c company_prod >  
/tmp/company_prod_${DATE}.backup
```

```
scp \
/tmp/company_prod_${DATE}.backup \
david@monitoring:/backups/postgresql/logical/
rm -f /tmp/company_prod_${DATE}.backup
```

Permissions :

```
chmod +x ~/scripts/postgres_logical_backup.sh
```

Test :

```
~/scripts/postgres_logical_backup.sh
```

```
root@pg-primary:/scripts# ./postgres_logical_backup.sh
Password:
gottlieb@monitoring's password:
company_prod_2026-06-14.backup
100% 19KB 6.6MB/s 00:00

root@monitoring:~# ll /backups/postgresql/logical/
total 48
drwxr-xr-x 2 gottlieb gottlieb 4096 juin 14 13:08 ./
drwxr-xr-x 5 gottlieb gottlieb 4096 juin 13 17:27 ../
-rw-r--r-- 1 gottlieb gottlieb 19123 juin 14 13:08 company_prod_2026-06-14.backup
-rw-r--r-- 1 gottlieb gottlieb 19123 juin 14 12:39 company_prod.backup
```

IX. Automated Scheduling

Create cron job :

```
crontab -e
```

Add :

```
0 1 * * * /home/david/scripts/postgres_logical_backup.sh
```

Meaning : Every day at 01:00

Verify :

```
crontab -l
```

```
root@pg-primary:/scripts# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
0 1 * * * /scripts/postgres_logical_backup.sh
```

X. Backup Retention Policy

On monitoring.

Create cleanup script :

```
nano ~/cleanup_backups.sh
```

Content :

```
#!/bin/bash
```

```
find \
/backups/postgresql/logical \
-type f \
-mtime +30 \
-delete
```

Permissions :

```
chmod +x cleanup_backups.sh
```

Schedule :

```
0 2 * * * /scripts/cleanup_backups.sh
```

XI. Physical Backup Strategy

Physical backups protect :

- Entire cluster
- System catalogs
- Roles
- Configuration

Create Physical Backup

On pg-primary :

```
pg_basebackup \
-h localhost \
-D /tmp/physical_backup \
-U backup_operator \
-P \
-R
```

```
root@pg-primary:~# pg_basebackup -h 192.168.10.21 -D /tmp/physical_backup -U backup_operator -P -R
Password:
23158/23158 kB (100%), 1/1 tablespace
```

Verify :

```
ls -lh /tmp/physical_backup
```

```
root@pg-primary:~# ls -lh /tmp/physical_backup/
total 216K
-rw-r----- 1 root root 225 juin 14 15:03 backup_label
-rw-r----- 1 root root 134K juin 14 15:03 backup_manifest
drwx----- 5 root root 4,0K juin 14 15:03 base
drwx----- 2 root root 4,0K juin 14 15:03 global
drwx----- 2 root root 4,0K juin 14 15:03 pg_commit_ts
drwx----- 2 root root 4,0K juin 14 15:03 pg_dynshmem
drwx----- 4 root root 4,0K juin 14 15:03 pg_logical
drwx----- 4 root root 4,0K juin 14 15:03 pg_multixact
drwx----- 2 root root 4,0K juin 14 15:03 pg_notify
drwx----- 2 root root 4,0K juin 14 15:03 pg_replslot
drwx----- 2 root root 4,0K juin 14 15:03 pg_serial
drwx----- 2 root root 4,0K juin 14 15:03 pg_snapshots
drwx----- 2 root root 4,0K juin 14 15:03 pg_stat
drwx----- 2 root root 4,0K juin 14 15:03 pg_stat_tmp
drwx----- 2 root root 4,0K juin 14 15:03 pg_subtrans
drwx----- 2 root root 4,0K juin 14 15:03 pg_tblspc
drwx----- 2 root root 4,0K juin 14 15:03 pg_twophase
-rw-r----- 1 root root 3 juin 14 15:03 PG-VERSION
drwx----- 3 root root 4,0K juin 14 15:03 pg_wal
drwx----- 2 root root 4,0K juin 14 15:03 pg_xact
-rw-r----- 1 root root 398 juin 14 15:03 postgresql.auto.conf
-rw-r----- 1 root root 0 juin 14 15:03 standby.signal
root@pg-primary:~#
```

Archive :


```
tar -czf \
physical_backup.tar.gz \
/tmp/physical_backup
```

Transfer :

```
scp \
physical_backup.tar.gz \
david@monitoring:/backups/postgresql/physical/
-rw-r--r-- 1 gottlieb gottlieb 19123 juin 14 12:39 company_prod.backup
root@monitoring:~# ll /backups/postgresql/physical/
total 3108
drwxr-xr-x 2 gottlieb gottlieb 4096 juin 14 15:10 ./
drwxr-xr-x 5 gottlieb gottlieb 4096 juin 13 17:27 ../
-rw-r--r-- 1 gottlieb gottlieb 3173139 juin 14 15:10 physical_backup.tar.gz
root@monitoring:~#
```

XII. PostgreSQL WAL Archiving

This is the foundation of Point-in-Time Recovery.

1. Create WAL Archive Directory

On pg-primary :

```
sudo mkdir -p /postgresql_archive
sudo chown postgres:postgres \
/postgresql_archive
```

2. Enable WAL Archiving

Edit :

```
sudo nano \
/etc/postgresql/*/main/postgresql.conf
```

Configure :

```
wal_level = replica
archive_mode = on
archive_command = 'cp %p /postgresql_archive/%f'
```

Restart :

```
sudo systemctl restart postgresql
```

Verify :

```
sudo -u postgres psql
SHOW archive_mode;
```

Generate activity :

```
INSERT INTO hr.employees
(first_name,last_name,department,hire_date)
VALUES
('Backup','Test','IT',NOW());
```

Verify WAL files :

```
ls -lh /postgresql_archive
```

```
root@pg-primary:~# ll /postgresql_archive/
total 16392
drwxr-xr-x  2 postgres postgres  4096 juin  14 15:42 ./
drwxr-xr-x 26 root      root      4096 juin  14 15:31 ../
-rw-----  1 postgres postgres 16777216 juin  14 15:42 00000001000000000000000002
root@pg-primary:~# ls -lh /postgresql_archive/
total 16M
-rw-----  1 postgres postgres 16M juin  14 15:42 00000001000000000000000002
root@pg-primary:~# █
```

XIII. Database Restore Exercise

A backup is useless if it cannot be restored.

1. Create Restore Database

```
sudo -u postgres psql
```

```
CREATE DATABASE company_restore_test;
```

2. Restore

```
pg_restore \
-h localhost \
-U dba_admin \
-d company_restore_test \
/db_backups/company_prod.backup
```

Verify :

```
sudo -u postgres psql
\c company_restore_test
SELECT * FROM hr.employees;
```

Expected :

Records successfully restored

```
company_restore_test=# SELECT * FROM hr.employees;
 id | first_name | last_name | department | hire_date
-----+-----+-----+-----+-----
  1 | John      | DOE      | IT         | 2025-01-01
(1 row)

company_restore_test=# █
```

XIV. Disaster Recovery

1. Scenario 1

i. Simulate Accidental Deletion

Connect :

```
DELETE FROM hr.employees;
```

Verify :

```
SELECT * FROM hr.employees;
```

```
root@pg-primary:~# sudo -u postgres psql
psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# \c company_prod
You are now connected to database "company_prod" as user "postgres".
company_prod=# DELETE FROM hr.employees;
DELETE 3
company_prod=# SELECT * FROM hr.employees;
 id | first_name | last_name | department | hire_date
-----+-----
(0 rows)

company_prod=#
```

ii. Recovery

Drop restore database :

```
DROP DATABASE company_restore_test;
```

Recreate :

```
CREATE DATABASE company_restore_test;
```

Restore backup :

```
pg_restore \
-h localhost \
-U dba_admin \
-d company_restore_test \
/db_backups/company_prod.backup
```

```
company_restore_test=# \c company_restore_test
You are now connected to database "company_restore_test" as user "postgres".
company_restore_test=# SELECT * FROM hr.employees;
 id | first_name | last_name | department | hire_date
-----+-----
 1 | John      | DOE       | IT          | 2025-01-01
(1 row)

company_restore_test=#
```

2. Scenario 2

i. Simulate Table Deletion

```
DROP TABLE inventory.products;
```

Attempt query :

```
SELECT * FROM inventory.products;
```

```
company_restore_test=# \c company_prod
You are now connected to database "company_prod" as user "postgres".
company_prod=# DROP TABLE inventory.products;
DROP TABLE
company_prod=# SELECT * FROM inventory.products;
ERROR: relation "inventory.products" does not exist
LINE 1: SELECT * FROM inventory.products;
               ^
company_prod=#
```

ii. Restore database

```
postgres=# \c company_restore_test
You are now connected to database "company_restore_test" as user "postgres".
company_restore_test=# SELECT * FROM inventory.products;
 product_id | product_name | quantity
-----
1 | Laptop | 20
(1 row)
```

XV. Backup Validation Procedure

A professional DBA always validates backups.

Weekly checklist :

Validation	Status
Backup Created	☐
Backup Copied	☐
Backup Integrity Verified	☐
Restore Tested	☐
WAL Archive Active	☐
Cleanup Executed	☐

XVI. Backup Monitoring

Useful commands.

- Check archive mode
- *SHOW archive_mode;*
- Check WAL activity

```
SELECT *
FROM pg_stat_archiver;
```

- Check backup storage
- *du -sh /backups/postgresql/**
- Check backup age

```
find /backups/postgresql \
-type f \
-mtime -7
```

XVII. Enterprise Backup Policy

1. Logical Backups

- Frequency : Daily
- Retention : 30 Days

2. Physical Backups

- Frequency : Weekly
- Retention : 12 Weeks

3. WAL Archives

- Retention : 14 Days

4. Restore Tests

- Frequency : Monthly